

PROGRAMAREA CALCULATOARELOR ȘI LIMBAJE DE PROGRAMARE

Tema #3 Tablouri, alocare dinamică, șiruri de caractere, structuri și fișiere.

Deadline: 15.01.2026

Responsabili temă: Alex Deonise, Cosmin-Ștefan Popa, Vlad-Andrei Chira

Changelog: Vă rugăm să adresați toate întrebările legate de această temă pe forumul asociat temei 3 de pe site-ul de curs. Orice clarificare / corectare / actualizare legată de enunț, checker sau lucruri administrative etc, vor fi anunțate exclusiv pe forumul dedicat temei 3.

- 16.12.2025: Publicare tema 3.
Această temă are un unic deadline (hard). Nu se acceptă submitii după data de 15 ianuarie.

Contents

Introducere	3
Obiectivul temei	3
Funcționarea programului	3
Undo și redo	3
Formatul unui fișier PPM	4
Exemplu	5
Grafeme fractale	5
Formatul unui fișier .lsys	6
Exemple	7
Exemplul 1	7
Exemplul 2	7
Exemplul 3	8
Turtle graphics	8
Algoritmul lui Bresenham pentru linii	10
Pseudocod	11
Exemple	12
Exemplul 1	12
Exemplul 2	13
Exemplul 3	13
Font	14
Formatul BDF	15
Descrierea unui caracter	15
Exemplu de fișier BDF	17
Exemple	18
Exemplul 1	18
Exemplul 2	18
Gravură	19
Exemple	19
Regulament	21
Arhivă	21
Checker	21
Punctaj	21
Reguli și precizări	22
Alte precizări	22

Introducere

Ceasul de 1000 de ani a sunat din nou pentru reînnoirea runelor străvechi. Mii de pietre runice s-au estompat și trebuie regenerate. Altfel, pădurea se va întuneca pe veci.

Sarcina seculară îi revine tot lui Gregor, ultimul vrăjitor ce cunoaște taina pietrelor. Totuși, având un calculator și un compilator de C, există o cale de a-l ajuta pe bătrânul Gregor: runele imită structuri fractale și le putem reproduce cu algoritmi lingvistici.

În cadrul temei vom implementa un program interactiv care citește comenzi, efectuează operațiile corespunzătoare asupra unei imagini și afișează rezultatul.

Dorim să încărcăm și să salvăm imagini ca fișiere. În acest scop, vom folosi formatul binar pentru imagini color **PPM**. Documentația formatului se găsește la adresa:

- <https://netpbm.sourceforge.net/doc/ppm.html>

Obiectivul temei

Prin această temă urmărim în principal:

- lucrul cu tablouri, alocare dinamică și structuri de date simple (inclusiv stocarea stării pentru comenzile UNDO și REDO);
- exersarea deep copy-ului stării programului pentru operații de tip undo/redo;
- lucrul cu fișiere binare (imagini PPM citite/scrise cu fread/fwrite);
- lucrul cu fișiere text (formatele .lsys și .bdf);
- reprezentarea binară a datelor (taskul de "gravură"/comanda BITCHECK).

Funcționarea programului

Programul va porni fără parametri în linia de comandă și va permite introducerea la `stdin` unor comenzi, câte una pe linie. Pentru fiecare comandă, programul va efectua operația cerută, după care va afișa pe ecran un mesaj (de succes sau de eroare). Mesajele ce vor fi afișate sunt specificate în descrierea comenzilor.

Comenzile suportate vor fi introduse la fiecare task. Comenzile se vor citi și rula până la întâlnirea comenzii `EXIT`, care oprește execuția programului, cu eliberarea tuturor resurselor și închiderea fișierelor.

O linie care nu poate fi interpretată ca una dintre comenzile definite la fiecare task va avea ca rezultat afișarea mesajului `Invalid command` și nu va genera alte efecte.

Undo și redo

Comenzile care modifică cu succes starea programului trebuie să poată fi anulate. Numim aceste comenzi **undoable**. Comanda `UNDO` va anula efectul ultimei comenzi **undoable** care nu a fost anulată. Prin anularea efectului înțelegem că pentru **orice comandă undoable *x*** și **orice altă comandă *y***, secvența:

```
1 X
2 UNDO
3 Y
```

trebuie să aibă același rezultat ca secvența:

```
1 Y
```

Dacă între o comandă undoable și UNDO sunt alte comenzi non-undoable, nu se va încerca anularea lor, ci vor rămâne executate. Spre exemplu, în urma secvenței (unde SAVE **nu** este undoable):

```
1 X
2 SAVE img1.ppm
3 UNDO
4 Y
5 SAVE img2.ppm
```

programul va salva o imagine `img1.ppm`, în care este vizibil efectul comenzii `X`, și o imagine `img2.ppm`, în care este vizibil efectul comenzii `Y` (dar nu și efectul comenzii `X`).

Dacă nu s-a executat **măcar o comandă undoable** (care nu a fost anulată) după începerea programului, comanda UNDO va eșua cu mesajul `Nothing to undo`.

De asemenea, comanda REDO va executa ultima comandă anulată. Spre exemplu, secvența:

```
1 X
2 UNDO
3 SAVE img.ppm
4 REDO
```

are același efect ca secvența:

```
1 SAVE img.ppm
2 X
```

Dacă nu există nicio "ultimă comandă anulată" sau dacă s-au executat **alte comenzi undoable** după ultima comandă anulată, comanda REDO va eșua cu mesajul `Nothing to redo`.

Formatul unui fișier PPM

Un fișier `.ppm` din această temă este un fișier binar în format PPM, varianta P6. Headerul (primele linii) este în text ASCII, iar matricea de pixeli este stocată binar, ca o succesiune de octeți.

Fișierele ce codifică imagini vor avea următoarea structură:

- Prima linie conține semnătura (*magic bytes*) care identifică formatul PPM: cele două caractere `P6`.
- Pe a doua linie se află cele 2 dimensiuni ale imaginii, în format ASCII.
- A treia linie conține valoarea maximă a unui canal (în cazul nostru, 255).
- Începând de la linia următoare, sunt reprezentate unul după altul cele h rânduri ale matricii de pixeli, de sus în jos.
 - Fiecare rând cuprinde w pixeli codificați în ordine, de la stânga la dreapta.
 - * Cele trei canale R , G și B ale unui pixel sunt reprezentate în baza 2 cu câte un octet.

În zona de pixeli, octeții apar în următoarea ordine:

- se iau rândurile imaginii de sus în jos;
- în fiecare rând, pixelii apar de la stânga la dreapta;
- pentru fiecare pixel, ordinea octeților este R , apoi G , apoi B .

De exemplu, dacă avem o imagine cu lățimea w și înălțimea h , zona de pixeli începe cu octeții corespunzători pixelului de la coordonatele $(0, h - 1)$, apoi $(1, h - 1)$, ..., $(w - 1, h - 1)$, continuă cu rândul următor și se termină cu pixelul $(w - 1, 0)$.

Cum nu toate valorile de la 0 la 255 sunt coduri ASCII ale unor caractere printabile, aceste fișiere nu se pot vizualiza corect cu un editor de text. În schimb, putem folosi programul **hexdump** pentru a le studia.

Comanda `hexdump -C <fișier>` afișează:

- Pe prima coloană, offsetul din fișier, în byți, la care începe rândul curent;
- Pe coloana din mijloc, conținutul fișierului (fiecare octet este codificat cu două cifre hexazecimale)
- În partea dreaptă, interpretarea în ASCII a octeților din rând. Pentru valorile `'\n'` sau care nu reprezintă caractere printabile în ASCII se afișează caracterul punct.

Observație: outputul comenzii `hexdump` afișează octeții în reprezentare hexazecimală, doar pentru vizualizare și debugging; în fișier ei sunt stocați ca octeți binari, nu ca text cu cifre hexazecimale.

Exemplu

```

1 % hexdump -C image.ppm
2 00000000  50 36 0a 33 20 32 0a 32  35 35 0a ff 00 00 00 ff  |P6.3 2.255.....|
3 00000010  00 00 00 ff ff ff 00 ff  ff ff 00 00 00  |.....|
4 0000001d

```

Listing 1: Hexdump al unei imagini color formate din 6 pixeli.

În Listing 1 am inclus un hexdump al imaginii din Figura 1.

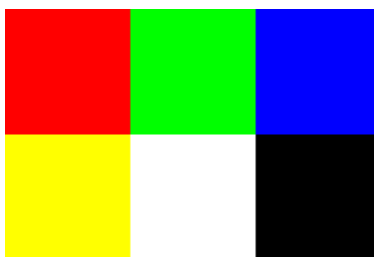


Figure 1: Imaginea codificată de fișierul din Listing 1 (mărită).

Sursa imaginii: [Wikipedia - Netpbm](#)

Grafeme fractale

La baza runelor bătrânei păduri stau șiruri de simboluri, incantații scrise într-un alfabet uitat. Gregor știe că înainte să poți obține runele, trebuie să stăpânești limba din care se nasc.

Definim un sistem Lindenmayer (sau L-system) notat G ca un triplet (V, ω, P) , unde:

- V (alfabetul) este o mulțime finită de simboluri
- ω este **axioma** sistemului, un șir de simboluri din V
- P e o mulțime de **reguli de producție** de forma $a \rightarrow \chi$, unde $a \in V$ iar χ (**succesorul** simbolului a) este un șir de simboluri din V .

În cazul nostru, alfabetul V conține caracterele ASCII printabile ce pot fi reprezentate de tipul `char` (litere, cifre și simboluri speciale, dar fără spații).

Fie $G = (V, \omega, P)$ un L-system. Dacă $\mu = a_1 a_2 \dots a_r$ e un șir de simboluri din V și există regulile de producție $a_1 \rightarrow \chi_1, \dots, a_r \rightarrow \chi_r$ în mulțimea P , atunci spunem că șirul $\mu' = \chi_1 \chi_2 \dots \chi_r$ (concatenarea șirurilor $\chi_1, \dots, \chi_r$) **derivă din** μ și scriem $D_G(\mu) = \mu'$. Dacă pentru un simbol $a \in V$ nu există nicio regulă de producție $a \rightarrow \chi$ în P , considerăm implicită regula $a \rightarrow a$ (simbolul rămâne neschimbat prin derivare).

Șirul μ' poate fi derivat din nou, înlocuind fiecare simbol ce alcătuiește μ' cu succesorul său, și se obține $\mu'' = D_G(\mu') = D_G^2(\mu)$, a doua derivare a șirului μ . În general, pentru orice șir μ și număr natural n , numim $D_G^n(\mu) = \underbrace{D_G(D_G(\dots D_G(\mu) \dots))}_{n \text{ ori}}$ a **n -a derivare** a șirului μ , cu convenția $D_G^0(\mu) = \mu$. În particular, $D_G^n(\omega)$ se numește a **n -a derivare a sistemului G** .

Exemplu

Fie un L-system $G = (V, \omega, P)$, unde:

- $V = \{A, B\}$
- $\omega = A$
- $P = \{A \rightarrow AB, B \rightarrow A\}$

Aplicăm derivarea pas cu pas asupra axiomei:

- $D_G^0(\omega) = A$
- $D_G^1(\omega) = D_G(A) = AB$
- $D_G^2(\omega) = D_G(AB) = AB A$
- $D_G^3(\omega) = D_G(ABA) = AB A AB$
- ...

Comenzile necesare pentru acest task sunt:

- **LSYSTEM <fișier>**
 - Va încărca în memoria programului un L-system descris în fișierul dat.
 - Orice alt L-system deja încărcat va fi înlocuit.
 - Comanda va afișa:
 - Loaded <fișier> (L-system with <nrules> rules) dacă operația s-a executat cu succes, unde:
 - <nrules> este numărul de reguli de producție din L-system.
 - Failed to load <fișier>, dacă operația a întâmpinat dificultăți.
 - **Comanda este undoable** în caz de succes.
- **DERIVE <n>**
 - Va produce și va afișa la stdout a <n>-a derivare a L-system-ului curent.
 - Va afișa No L-system loaded dacă nu există niciun L-system încărcat în memorie.

Formatul unui fișier .lsys

Un fișier .lsys este un fișier text (ASCII) cu următorul format:

- Pe prima linie a unui fișier .lsys se va regăsi axioma sistemului, un șir de caractere fără spații.
- Pe a doua linie se va regăsi un număr întreg mai mare sau egal cu 0, nrules, ce reprezintă numărul de reguli de producție.
- Următoarele nrules linii conțin câte o regulă pe fiecare linie.
- O regulă este de forma <simbol> <succesor>, unde:
 - <simbol> este un singur caracter (diferit de spațiu)
 - <succesor> este un șir de caractere fără spații

Exemple

Fie fișierul `simple.lsys` care conține:

```
1 F
2 1
3 F F+F-F
```

Iar fișierul `fib.lsys` conține:

```
1 A
2 2
3 A AB
4 B A
```

Exemplul 1

Date de intrare

```
1 LSYSTEM simple.lsys
2 DERIVE 0
3 DERIVE 1
4 EXIT
```

Date de ieșire

```
1 Loaded simple.lsys (L-system with 1 rules)
2 F
3 F+F-F
```

Explicație

Axioma este `F`, cu o singură regulă `F -> F+F-F`.

`DERIVE 0` afișează axioma.

`DERIVE 1` înlocuiește `F` cu `F+F-F`.

Exemplul 2

Date de intrare

```
1 LSYSTEM no_such_file.lsys
2 DERIVE 1
3 EXIT
```

Date de ieșire

```
1 Failed to load no_such_file.lsys
2 No L-system loaded
```

Explicație

Fișierul `no_such_file.lsys` nu există sau nu poate fi deschis/parsat. Comanda `LSYSTEM` eșuează și programul afișează: `Failed to load no_such_file.lsys`. Starea rămâne „fără L-system încărcat”, deci `DERIVE` 1 va afișa `No L-system loaded`.

Exemplul 3

Date de intrare

```

1 LSYSTEM fib.lsys
2 DERIVE 0
3 DERIVE 1
4 DERIVE 2
5 DERIVE 3
6 DERIVE 4
7 EXIT

```

Date de ieșire

```

1 Loaded fib.lsys (L-system with 2 rules)
2 A
3 AB
4 ABA
5 ABAAB
6 ABAABABA

```

Turtle graphics

Runele au fost regenerate în memorie, dar pentru piatra pădurii asta nu e de ajuns. Piatra nu înțelege simboluri și reguli, ci doar urme de culoare: linii, unghiuri, ramuri. De aceea Gregor apelează la o mică țestoasă magică.

Putem asocia simbolurile obținute prin derivarea unui L-system unor diferite acțiuni. În acest task, asociem simbolurile cu instrucțiuni destinate unei țestoase care desenează linii pe imagini.

Definim întâi un **sistem grafic**: este un triplet (I, T, S) ce cuprinde o imagine I , starea T a țestoasei și un vector S .

Imaginea este reprezentată de o matrice de pixeli cu lățimea w și înălțimea h . Fiecare pixel e identificat de două coordonate (x, y) , numere întregi, cu coordonata $(0, 0)$ în colțul din **stânga jos** al imaginii. Un pixel este un triplet (R, G, B) ce conține valori de la 0 la 255 pentru fiecare din cele trei canale.

Imaginile suportă operația `draw_line( $I, p_0, p_1, color$ )` – desenarea unui segment de dreaptă de culoare `color` de la punctul p_0 la p_1 .

Atenție!

Imaginile sunt discrete; coordonatele reale trebuie convertite la coordonate întregi înainte de a fi utilizate în operații asupra unei imagini.

Țestoasa există într-un sistem de coordonate carteziane xOy și are o stare dată de:

- **poziție** – o pereche cu **elemente reale** $p = (x, y)$,

- **orientare** – un unghi θ față de axa Ox pozitivă (determină direcția în care se îndreaptă țestoasa).

Țestoasa se poate deplasa în spațiul xOy , iar mișcarea acesteia este caracterizată de:

- **pas de deplasare** – o lungime $d > 0$,
- **pas unghiular** – un unghi $\delta \in (0, 90]$.

Vectorul S este folosit pentru a salva și restaura starea țestoasei, și suportă trei operații:

- `add_state( $S, (p, \theta)$ )` – inserează poziția p și orientarea θ **la sfârșitul vectorului S** .
- `get_state( $S$ )` – returnează poziția și orientarea aflate **la sfârșitul vectorului S** .
- `remove_state( $S$ )` – elimină poziția și orientarea **de la sfârșitul vectorului S** .

Tabelul de mai jos descrie simbolurile asociate cu instrucțiuni pentru țestoasă. Dacă un parametru al sistemului grafic nu e menționat, înseamnă că rămâne neschimbat sub acțiunea comenzii.

Simbol	Comandă
F	Țestoasa înaintază cu d unități pe direcția actuală definită de poziție și orientare. Mișcarea țestoasei corespunde desenării pe imaginea I a unei linii de grosime 1 pixel de la poziția actuală la noua poziție; Noua imagine se numește I' .
+	Orientarea țestoasei crește cu un pas unghiular: $\theta' = \theta + \delta$.
-	Orientarea țestoasei scade cu un pas unghiular: $\theta' = \theta - \delta$.
[	Poziția și orientarea curente p, θ ale țestoasei se inserează la sfârșitul lui S , deci $S' = \text{add_state}(S, (p, \theta))$.
]	Țestoasa revine la poziția și orientarea care se extrag de la sfârșitul lui S : $(p', \theta') = \text{get_state}(S)$ și $S' = \text{remove_state}(S)$. Această operație nu implică desenarea unei linii.
alt simbol	Sistemul rămâne neschimbat.

În exemplul din Figura 2, imaginea inițială are toți pixelii albi, țestoasa pornește din partea de jos a imaginii cu orientarea 90° , și se obține reprezentarea grafică a șirului $F[+F]-F$. Pasul unghiular este 45° .

F [+ F] - F

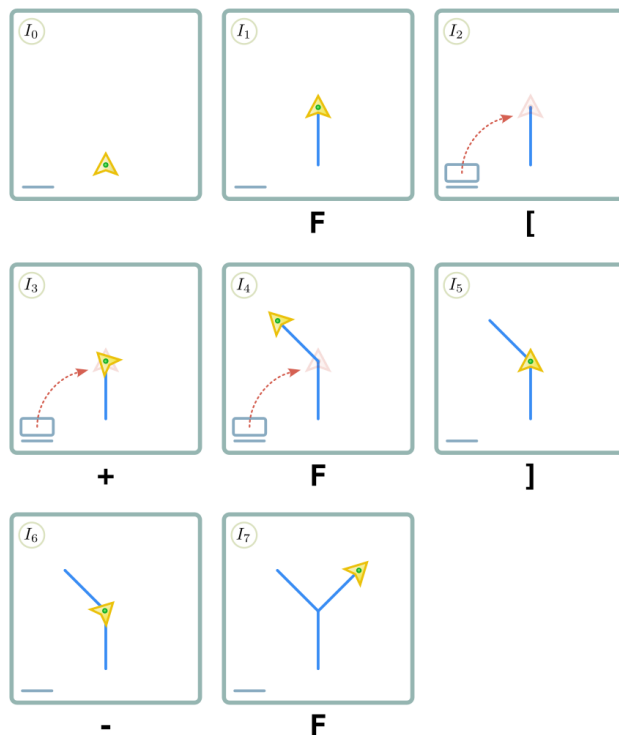


Figure 2: Evoluția sistemului grafic la interpretarea simbolurilor ca operații.

Algoritmul lui Bresenham pentru linii

Deoarece o linie este o construcție geometrică continuă, aceasta nu poate fi reprezentată în mod discret fără o aproximare. În cazul desenării unei linii pe o imagine, trebuie să alegem un șir de pixeli astfel încât acesta să urmeze cât mai bine linia matematică dintre cele două puncte.

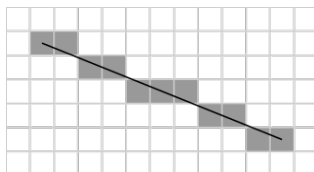


Figure 3: Rezultatul algoritmului lui Bresenham: o linie de la poziția (1, 5) la (11, 1).

Sursa imaginii: [Wikipedia – Bresenham's line algorithm](#)

Algoritmul lui Bresenham folosește următoarele idei:

1. Linia se parcurge de la un capăt la altul, algoritmul alegând succesiv pixelul următor ce aparține liniei.
2. La fiecare pas, algoritmul decide dacă următorul pixel ce trebuie ales se află:
 - Doar pe orizontală/verticală (caz în care algoritmul se deplasează cu 1 pixel pe axa dominantă)
 - Pe diagonală (caz în care algoritmul se deplasează pe ambele axe)
3. Algoritmul păstrează o **eroare** - o mărime ce descrie abaterea de la linia continuă. Când eroarea depășește o anumită limită, algoritmul se deplasează și pe cealaltă axă pentru a corecta eroarea.

Pseudocod

Notă

Pseudocodul de mai jos este doar o posibilă variantă a algoritmului lui Bresenham. Nu sunteți constrânși să îl implementați identic; orice implementare corectă a algoritmului este acceptată.

Toate variabilele din pseudocodul de mai jos sunt de tip număr întreg.

```

1 function draw_line(x0, y0, x1, y1):
2     dx = abs(x1 - x0)
3     sx = 1 if x0 < x1 else -1
4
5     dy = -abs(y1 - y0)
6     sy = 1 if y0 < y1 else -1
7
8     err = dx + dy
9     loop:
10        draw_pixel(x0, y0)
11
12        if x0 == x1 and y0 == y1: // Capatul liniei
13            break
14
15        e2 = 2 * err
16
17        if e2 >= dy:
18            err = err + dy
19            x0 = x0 + sx
20        if e2 <= dx:
21            err = err + dx
22            y0 = y0 + sy
  
```

Comenzile necesare pentru acest task sunt:

- LOAD <fisier>
 - Va încărca în memoria programului imaginea PPM din fișierul dat.
 - Orice imagine deja încărcată în memorie va fi înlocuită.
 - Outputul comenzii va fi:
 - Loaded <fisier> (PPM image <width>x<height>) dacă operația s-a executat cu succes, unde
 - <width>, <height> sunt lățimea, respectiv înălțimea imaginii încărcate.
 - Failed to load <fisier>, dacă operația a întâmpinat dificultăți.
 - **Comanda este undoable** în caz de succes.
- SAVE <fisier>
 - Salvează imaginea curentă în fișierul <fisier> folosind formatul PPM.
 - Outputul comenzii va fi:
 - Saved <fisier> dacă imaginea a fost salvată, sau
 - No image loaded, dacă încă nu s-a încărcat o imagine.
- TURTLE <x> <y> <pas_deplasare> <orientare> <pas_unghiular> <n> <R> <G>
 - Va crea un sistem grafic cu imaginea curentă, folosind parametrii inițiali pentru testoașă (poziția (<x>, <y>) și orientarea inițială) și pașii (de deplasare și unghiular), și va interpreta simbolurile

cele de-a <n>-a derivări a L-systemului activ, modificând corespunzător imaginea curentă. Liniile sunt desenate cu culoarea (<R>, <G>,). Toate unghiurile sunt date în grade.

- Outputul comenzii va fi:
 - Dacă nu există nicio imagine încărcată, se afișează `No image loaded`
 - Dacă nu există un L-system încărcat, se afișează `No L-system loaded.`
 - În caz de succes, se afișează `Drawing done.`
- **Comanda este undoable** în caz de succes.

Exemple

Fie fișierul `line.lsys`:

```
1 F
2 0
```

Fie fișierul `branch.lsys`:

```
1 F
2 1
3 F F[+F]-F
```

Fie fișierul `plant.lsys`:

```
1 X
2 2
3 X F-[[X]+X]+F[+FX]-X
4 F FF
```

Exemplul 1

Date de intrare

```
1 LSYSTEM line.lsys
2 LOAD blank_200x200.ppm
3 TURTLE 100 100 40 90 0 0 255 0 0
4 SAVE line.ppm
5 EXIT
```

Date de ieșire

```
1 Loaded line.lsys (L-system with 0 rules)
2 Loaded blank_200x200.ppm (PPM image 200x200)
3 Drawing done
4 Saved line.ppm
```

Explicație

`line.lsys` are axioma `F` și 0 reguli, deci derivarea cu numărul 0 este chiar `F`. Comanda `TURTLE 100 100 40 90 0 0 255 0 0` pornește țestoasa din centrul imaginii (100, 100), cu pas de deplasare 40, orientare 90° (în sus), fără rotiri, și desenează o singură linie verticală de culoare roșie.

Exemplul 2**Date de intrare**

```

1 LSYSTEM branch.lsys
2 LOAD blank_400x400.ppm
3 TURTLE 200 50 50 90 45 1 10 10 255
4 SAVE branch.ppm
5 EXIT

```

Date de ieșire

```

1 Loaded branch.lsys (L-system with 1 rules)
2 Loaded blank_400x400.ppm (PPM image 400x400)
3 Drawing done
4 Saved branch.ppm

```

Explicație

branch.lsys are axioma F și regula $F \rightarrow F[+F]-F$. Cu $nr_derivare = 1$, șirul devine $F[+F]-F$. Testoasa pornește de jos, centrată pe orizontală, cu unghi inițial 90° , pas de deplasare 50 și pas unghiular 45° , rezultând un trunchi vertical și două ramuri simetrice (Figura 2).

Exemplul 3**Date de intrare**

```

1 LSYSTEM plant.lsys
2 LOAD blank_800x800.ppm
3 TURTLE 400 50 8 90 25 5 0 55 0
4 SAVE plant.ppm
5 EXIT

```

Date de ieșire

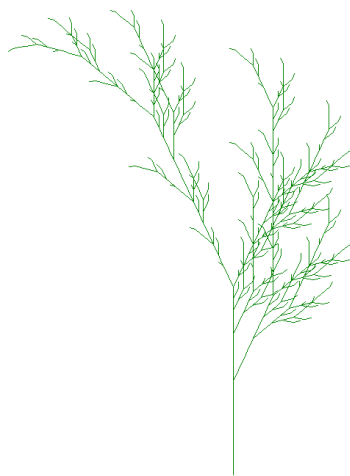
```

1 Loaded plant.lsys (L-system with 2 rules)
2 Loaded blank_800x800.ppm (PPM image 800x800)
3 Drawing done
4 Saved plant.ppm

```

Explicație

plant.lsys descrie L-systemul cu axioma X și regulile $X \rightarrow F-[X]+X]+F[+FX]-X$ și $F \rightarrow FF$. Testoasa pornește de la $(400, 50)$, cu unghi inițial 90° , pas de deplasare 8 pixeli și pas unghiular 25° , și desenează a cincea derivare a sistemului, obținând arborele fractal din Figura 4.

Figure 4: Imaginea plant.ppm generată în [Exemplul 3](#).

Font

Nimic nu e cu adevărat legat de lume până nu primește un nume. Gregor știe că a rosti numele unui lucru înseamnă a-i atinge esența, iar a-l scrie înseamnă a-l fixa în materie.

Mai întâi este nevoie să încărcăm un font (tip de literă) cu care vom reprezenta grafic caracterele ce alcătuiesc un mesaj text. Vom aplica fonturi **bitmap**, care definesc orice caracter tipografic (**glyph**) ca o matrice de pixeli (în cazul nostru, un pixel poate fi doar "absent" sau "prezent", deci matricea conține valori de 0 și 1).

În particular, folosim o descriere *human-readable* a fonturilor bitmap numită **Glyph Bitmap Distribution Format** (BDF). Documentația completă a formatului se găsește [aici](#).

Comenzile necesare pentru acest task sunt:

- FONT <fisier>
 - Încarcă un font bitmap din fișierul BDF <fisier> și îl setează ca font curent.
 - Orice font deja încărcat este eliberat și înlocuit.
 - Mesaje:
 - Loaded <fisier> (bitmap font <name>) dacă operația reușește, unde <name> este numele fontului (atributul FONT din fișierul BDF).
 - Failed to load <fisier> dacă fișierul nu poate fi deschis sau nu respectă formatul așteptat.
 - În caz de eșec, comanda nu are alte efecte.
 - **Comanda este undoable** în caz de succes.
- TYPE "<string>" <start_x> <start_y> <R> <G>
 - Desenează șirul de caractere <string> pe imaginea curentă, începând de la poziția (<start_x>, <start_y>), folosind:
 - fontul curent (încărcat anterior prin FONT),
 - culoarea de umplere (<R>, <G>,) pentru text (valori întregi între 0 și 255).
 - <string> este delimitat de ghilimele duble "<string>" și poate conține spații.
 - Se garantează că toate caracterele din <string> vor avea un glyph corespunzător în fontul încărcat.
 - Mesaje:

- No image loaded dacă nu există nicio imagine curentă (nu s-a apelat cu succes LOAD înainte).
- No font loaded dacă nu există niciun font curent (nu s-a apelat cu succes FONT înainte).
- Text written dacă tot textul a fost desenat pe imagine cu succes.
- **Comanda este undoable** în caz de succes.
- Comenzile LOAD și SAVE introduse la taskul anterior.

Formatul BDF

Un fișier .bdf este un fișier text (ASCII) în format Glyph Bitmap Distribution Format (BDF), așa cum este definit de Adobe. Un astfel de fișier conține o înșiruire de itemi, fiecare pe câte o linie.

Itemii au forma `ATRIBUT <argumente>`, unde argumentele pot fi de tipul `string` sau `integer` și sunt separate prin spații.

Structura generală a unui fișier BDF este următoarea:

```
1 STARTFONT <versiune format>
2 ... (diverse attribute)
3 STARTPROPERTIES <nprops>
4 ... (exact nprops proprietati optionale)
5 ENDPROPERTIES
6 CHARS <nglyphs>
7 ... (exact nglyphs descrieri de caracter)
8 ENDFONT
```

Secțiunea de proprietăți opționale este în sine opțională și o vom ignora dacă există.

De interes pentru această parte a temei sunt atributul `FONT` și descrierile caracterelor de după cuvântul cheie `CHARS`.

Atributul `FONT` apare după `STARTFONT` (înainte de `STARTPROPERTIES`), și este specificat astfel:

- `FONT <nume font>` – definește numele complet al fontului

Descrierea unui caracter

Fiecare descriere de caracter are următoarea structură. Pot exista alți itemi în interiorul descrierii (între `STARTCHAR` și `BITMAP`); îi vom ignora. De asemenea, **itemii pot apărea în orice ordine**.

```

1 STARTCHAR <nume caracter>
2 ENCODING <cod caracter>
3 DWIDTH <dwx> <dwy>
4 BBX <BBw> <BBh> <BBxoff> <BByoff>
5 BITMAP
6 ... (exact BBh linii de date hexazecimale)
7 ENDCHAR

```

Avem următoarele atribute:

- **STARTCHAR** <nume caracter> – marchează începutul descrierii unui caracter numit *nume caracter*
- **ENCODING** <cod caracter> – codul ASCII¹ al caracterului descris (număr întreg)
- **DWIDTH** <dw_x> <dw_y> – numărul (întreg) de pixeli cu care se mută cursorul după scrierea caracterului:
 - dw_x pixeli pe axa *Ox*
 - dw_y pixeli pe axa *Oy*
- **BBX** <BB_w> <BB_h> <BB_xoff> <BB_yoff> – *bounding box* – dimensiunile matricei de pixeli care descrie caracterul, respectiv poziția la care trebuie desenat colțul **din stânga jos** al caracterului, relativ la poziția cursorului (toate numere întregi):
 - BB_w – numărul de **coloane** ale matricei de pixeli
 - BB_h – numărul de **rânduri** ale matricei de pixeli
 - BB_xoff – deplasarea caracterului pe axa *Ox* față de origine
 - BB_yoff – deplasarea caracterului pe axa *Oy* față de origine
- **BITMAP** – cuvântul cheie **BITMAP** e urmat de BB_h linii; fiecare linie este o reprezentare în baza 16 a rândului corespunzător din matricea de pixeli completat la dreapta cu zerouri până la un multiplu de 8 biți.
 - Spre exemplu, rândul 110001₍₂₎ al unei matrice de pixeli cu 6 coloane e completat cu două zerouri la dreapta: 11000100₍₂₎ și encodat în fișier cu cifre hexazecimale: C4.
 - Prima linie de după **BITMAP** encodează rândul cel mai de sus al matricei de pixeli care definește caracterul.

¹Specificatia formatului menționează **PostScript Standard Encoding**, care pentru caracterele pe care le folosim este identic cu ASCII.

Exemplu de fișier BDF

Reproducem mai jos un exemplu de font cu un singur caracter (lowercase j). După STARTFONT apar mai multe atribute: COMMENT, FONT, SIZE și altele. Fontul nu conține STARTPROPERTIES/ENDPROPERTIES. Itemul CHARS 1 indică numărul de descrieri de caracter care urmează (una singură).

```

1 STARTFONT 2.1
2 COMMENT Font adaptat din specificatia formatului
3 FONT FontulMeu-Italic
4 SIZE 8 200 200
5 FONTBOUNDINGBOX 9 24 -2 -6
6 CHARS 1
7 STARTCHAR Unicul caracter j
8 ENCODING 106
9 SWIDTH 355 0
10 DWIDTH 8 0
11 BBX 9 22 -2 -6
12 BITMAP
13 0380
14 0380
15 0380
16 0380
17 0000
18 0700
19 0700
20 0700
21 0700
22 0E00
23 ... (12 randuri de bitmap omise)
24 ENDCHAR
25 ENDFONT

```

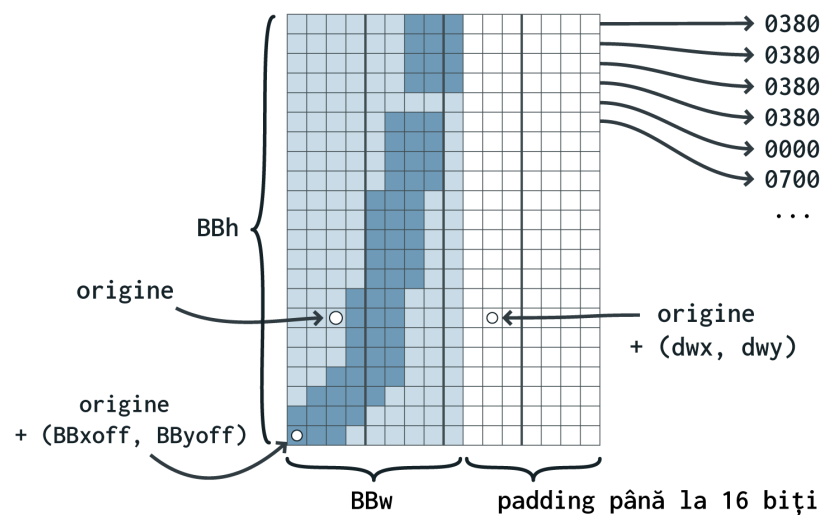


Figure 5: Atributele caracterului j din exemplu.

În Figura 5 găsim o reprezentare vizuală a descrierii de caracter. Zona umbrită marchează bounding box-ul caracterului. Bitmapul este mai larg – are lățimea egală cu BBw rotunjit în sus la un multiplu de opt biți. Pixelii colorați cu albastru închis în figură au valoarea 1 în bitmap; ceilalți au valoarea 0. Pozițiile cu valoarea 1 indică ce pixeli din imagine vor primi o culoare de umplere la desenarea caracterului.

Dacă un caracter trebuie desenat la o anumită poziție (x, y) , înseamnă că pixelii trebuie amplasați astfel încât **originea** caracterului să coincidă cu poziția (x, y) . Cu alte cuvinte, **colțul din stânga jos** al caracterului desenat se află la poziția $(x, y) + (BBxoff, BByoff)$. Următorul caracter din același text va fi desenat la poziția $(x, y) + (dwx, dwy)$.

Exemple

Exemplul 1

Date de intrare

```
1 LOAD blank_400x200.ppm
2 TYPE "Ancient runes awaken" 50 80 255 255 255
3 SAVE text.ppm
4 EXIT
```

Date de ieșire

```
1 Loaded blank_400x200.ppm (PPM image 400x200)
2 No font loaded
3 Saved text.ppm
```

Explicație

Imaginea este încărcată, dar nu a fost executată nicio comandă FONT înainte de TYPE, deci nu există font curent. Comanda TYPE eșuează și afișează No font loaded, fără să modifice imaginea. Comanda SAVE salvează imaginea nemodificată în text.ppm.

Exemplul 2

Date de intrare

```
1 LOAD blank_600x120.ppm
2 FONT mystic.bdf
3 TYPE "Whispering runes" 20 50 0 255 0
4 SAVE mystic.ppm
5 EXIT
```

Date de ieșire

```
1 Loaded blank_600x120.ppm (PPM image 600x120)
2 Loaded mystic.bdf (bitmap font Mystic-13)
3 Text written
4 Saved mystic.ppm
```

Explicație

Se încarcă imaginea și fontul bitmap mystic.bdf. Comanda TYPE desenează textul la poziția (20, 50) cu culoarea (0, 255, 0), folosind fontul curent. SAVE scrie rezultatul în mystic.ppm (Figura 6).

Figure 6: Imaginea mystic.ppm generată de comenzile din [Exemplul 2](#).

Gravură

Imaginea generată va fi trimisă prin cablu către mașina de gravat, servind drept șablon. Avem la dispoziție un adaptor care, însă, din cauza interferenței cu ceasul de 1000 de ani, poate citi greșit anumite semnale. Dorim să detectăm aceste potențiale erori.

Pentru imaginea curentă, vom analiza secvența de octeți ce codifică matricea de pixeli, ca în **Formatul de imagine**: pixelii din fiecare rând, unul după altul, sunt reprezentați folosind trei octeți pentru cele trei canale. În șirul de biți care alcătuiește această codificare, fiecare secvență 0010 poate fi citită greșit ca 0000, și fiecare secvență 1101 poate fi citită greșit ca 1111. Cu alte cuvinte, dacă doi biți identici b sunt urmați de bitul negat $\bar{b}$, urmat din nou de bitul b , adaptorul ar putea citi b în locul lui $\bar{b}$.

Pentru acest task, este necesară implementarea următoarei comenzi:

- BITCHECK
 - Identifică toate secvențele 0010 și 1101 din codificarea imaginii curente și afișează potențialele erori de citire.
 - Mesaje:
 - No image loaded dacă nu există o imagine curentă.
 - Altfel, Warning: pixel at ($\langle x \rangle$, $\langle y \rangle$) may be read as ($\langle R' \rangle$, $\langle G' \rangle$, $\langle B' \rangle$) pentru fiecare apariție a unei secvențe 0010 sau 1101. Trebuie afișate coordonatele pixelului din care face parte bitul care ar putea fi interpretat greșit, și valoarea (R' , G' , B') care ar putea fi citită în locul celei corecte (afișarea se face în baza 10).

Exemple

Să considerăm o imagine PPM cu lățimea 1 și înălțimea 2 (o coloană și două rânduri). Pixelul de sus are valoarea (0, 0, 1), iar pixelul de jos are valoarea (32, 255, 253). Dacă această imagine este salvată într-un fișier .ppm conform formatului descris mai sus, zona de pixeli din fișier va fi codificată prin următoarea secvență de biți (scrisă mai jos grupată pe octeți):

```
1 00000000 00000000 00000001 00100000 11111111 11111101
```

Prima secvență 0010 apare la granița dintre pixelul de sus și cel de jos. Bitul 1 se află în interiorul pixelului de sus, care ar putea fi interpretat astfel de adaptor:

```
1 00000000 00000000 00000000
```

Următoarea secvență cu risc de eroare, 0010, apare în interiorul pixelului de jos, care ar putea fi citit astfel:

```
1 00000000 11111111 11111101
```

Și așa mai departe. Comanda BITCHECK va afișa:

```
1 Warning: pixel at (0, 1) may be read as (0, 0, 0)
2 Warning: pixel at (0, 0) may be read as (0, 255, 253)
3 Warning: pixel at (0, 0) may be read as (32, 255, 255)
```

Regulament

Regulamentul general al temelor se găsește pe ocw (**Temele de casă**). Vă rugăm să îl citiți integral înainte de a continua cu regulile specifice acestei teme.

Arhivă

Soluția temei se va trimite ca o arhivă **zip**. Numele arhivei trebuie să fie de forma **Grupă_NumePrenume_TemaX.zip** - exemplu: 311CB_PopaCosmin_Tema3.zip.

Arhiva trebuie să conțină în directorul **RĂDĂCINĂ** doar următoarele:

- Codul sursă al programului vostru (fișierele **.c** și eventual **.h**).
- Un fișier **r** care să conțină regulile **build** și **clean**.
 - Regula **build** va compila codul vostru și va genera **doar** un singur executabil numit **runic**
 - Regula **clean** va șterge **toate** fișierele generate la build (executabile, binare intermediare etc).
- Un fișier **README** care să conțină prezentarea implementării alese de voi. **Nu** copiați bucăți din enunț.

Arhiva temei **NU** va conține: fișiere binare, fișiere de intrare/ieșire folosite de checker, checkerul, orice alt fișier care nu este cerut mai sus.

Numele și extensiile fișierelor trimise **NU** trebuie să conțină spații sau majuscule, cu excepția fișierelor README (care are numele scris cu majuscule și nu are extensie) și Makefile (care are doar prima literă majusculă).

Nerespectarea oricărei reguli din secțiunea **Arhivă** aduce un punctaj **NUL** (0) pe temă.

Checker

Pentru corectarea temei vom folosi scriptul **check** din arhiva **check_runic.zip** din secțiunea de resurse asociată temei. Vă rugăm să citiți **README-checker.md** pentru a ști cum să instalați și utilizați checker-ul.

Punctaj

Distribuirea punctajului:

- Implementarea L-systems: 9p
- Implementarea turtle graphics: 18p
- Implementarea textului pe imagini: 19p
- Text pe imagini și turtle graphics simultan: 18p
- Implementarea detectării erorilor (BITCHECK): 16p
- Claritatea și calitatea codului (inclusiv modularizare): 10p
- Claritatea explicațiilor din README: 10p

ATENȚIE! Punctajul maxim pe temă este **100p**. Acesta reprezintă 1.25p din nota finală la această materie. La această temă nu se vor acorda punctaje bonus. **O temă care nu folosește alocare dinamică NU va fi punctată.**

Reguli și precizări

- Punctajul pe teste este cel acordat de script-ul **check**, rulat pe **Moodle**. Echipa de corectare își rezervă dreptul de a depuncta pentru orice încercare de a trece testele fraudulos (de exemplu prin hardcodare, etc).
- Punctajul pe calitatea explicațiilor și a codului se acordă în mai multe etape:
 - **corectare automată**
 - * Checker-ul va încerca să detecteze în mod automat probleme legate de coding style și alte aspecte de organizare a codului.
 - * Acesta va puncta cu maxim 20p dacă nu sunt probleme detectate în mod automat.
 - * Punctajul se va acorda proporțional cu numărul de puncte acumulate pe teste din cele 80p.
 - * Checker-ul poate să aplice însă și penalizări (exemplu pentru warninguri la compilare) sau alte probleme descoperite la runtime.
 - **corectare manuală**
 - * Tema va fi corectată manual și se vor verifica și alte aspecte pe care checker-ul nu le poate prinde. Recomandăm să parcurgeți cu atenție tutorialul de **coding-style** de pe ocw.cs.pub.ro.
 - * Codul sursă trebuie să fie însoțit de un fișier README care trebuie să conțină informațiile utile pentru înțelegerea funcționalității, modului de implementare și utilizare a programului. Acesta evaluează, de asemenea, abilitatea voastră de a documenta complet și concis programele pe care le produceți și va fi evaluat, în mod analog coding-style-ului, de către echipa de asistenți. În funcție de calitatea documentației, se vor aplica depunctări sau bonusuri.
 - * La corectarea manuală se va acorda un bonus de maximum 10 puncte pentru modularizare. Deprinderea de a scrie cod sursă de calitate, este un obiectiv important al materiei. Sursele greu de înțeles, modularizate neadecvat sau care prezintă hardcodări care pot afecta semnificativ mentenabilitatea programului cerut, pot fi depunctate adițional. Penalizarea pentru modularizare defectuoasă sau absentă complet, poate să fie oricât de mare.
 - * În această etapă se pot aplica depunctări oricât de mari (chiar și totale). Orice rezultat / punctaj acordat automat de checker, poate fi anulat sau suprascris de către echipa de PCLP la corectarea manuală.
 - * O temă care **NU** compilează cu -Wall -Wextra este depunctată la corectarea manuală cu 5p (punctajul echivalent pentru warnings). Echipa de PCLP vă recomandă să compilați cu -Wall -Wextra -Werror.
 - * **Tema trebuie să reprezinte exclusiv munca studentului!** Echipa va aplica regulamentele în vigoare pentru situațiile de fraudă / plagiat, pentru orice nerespectare a acestor reguli, incluzând, dar nelimitându-se la: copierea de la colegi (se aplică pentru ambele persoane implicate) sau din alte surse a unor părți din temă, prezentarea ca rezultat personal a oricărei bucați de cod provenită din alte surse necitate sau neaprobată în prealabil (site-uri web, alte persoane, cod generat folosit AI / LLM-uri etc.).

Alte precizări

- Implementarea se va face în limbajul **C**, iar tema va fi compilată și testată **DOAR** într-un mediu **LINUX**. Nerespectarea acestor reguli aduce un punctaj **NUL**.
- Tema trebuie trimisă sub forma unei arhive pe site-ul cursului **curs.upb.ro**.
- Tema poate fi trimisă de oricâte ori fără depunctări până la deadline. Mai multe detalii se găsesc în regulamentul de pe [ocw](http://ocw.cs.pub.ro).
- O temă care **NU** compilează **NU** va fi punctată.
- O temă care compilează, dar care **NU** trece niciun test **NU** va fi punctată.
- Punctajul pe teste este cel acordat de **check** rulat pe **platforma remote a echipei de PCLP**. Echipa de corectare își rezervă dreptul de a depuncta pentru orice încercare de a trece testele fraudulos (de exemplu prin hardcodare).
- Ultima temă submitată pe Moodle poate fi rulată de către responsabili / echipa de PCLP de mai multe ori în vederea verificării faptului că nu aveți bug-uri în sursă. În cazul obținerii punctajelor diferite la rulări multiple, vom păstra minimul dintre punctaje. Vă recomandăm să verificați local tema de

mai multe ori pentru a verifica că punctajul este mereu același, apoi să încărcați tema. De asemenea, verificați că punctajul de pe platformă este cel așteptat - singurul punctaj / feedback relevant este cel de pe platforma remote PCLP (**vmchecker-next**).